Singly Linked List – Traverse List ReverseOrderIterative – Space Complexity and Algorithm.

Program:

```
void TraverseListReverseOrderIterative(Node **head)

{
    if (*head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    reverseList(head);
    Node **current = head;
    while (*current != nullptr)
    {
        cout << (*current)->data << " ";
        current = &((*current)->next);
    }
    cout << endl;
    reverseList(head);
}
....

        case 24: {

            cout << endl;
            cout << endl;
            cout << "Traverse List Reverse Order Iterative" << endl;
            TraverseListReverseOrderIterative(&head);
            cout << endl;
            cout << endl;
            break;
        }
```

Space Complexity Analysis

1. *Input Space Complexity*

A. Parameter – Only Definition

The input space complexity refers to the space required for the input parameters of the function.

In this case, the function takes:

- ***A pointer to a pointer (Node ** head), which requires a constant amount of space, O(1).***

Thus, the total input space complexity is: O(1).

• Node **head – one pointer-to-pointer (single machine word).

This view only considers the memory for the parameters passed to the function. The single parameter `Node **head` is a pointer-to-a-pointer, which has a **fixed, constant size**. Therefore, the input space complexity is **O(1)**.

B. Full – data (inclusive) Definition:

• Same single word as above, plus every node reachable from *head at call time.

• If the list contains n nodes, they already occupy Θ(n) bytes/words.

This more common definition includes the entire data structure. If the linked list has **n nodes**, the space required to store that list is proportional to `n`. Therefore, the input space complexity is **Θ(n)**.

2. *Auxiliary Space Complexity*

Auxiliary space complexity refers to the extra space used by the algorithm that is not part of the input. In the case of the TraverseListReverseOrderIterative function, the auxiliary space is determined by the variables declared within the function:

Memory this call stack consumes beyond the input:

• In TraverseListReverseOrderIterative – Node **current → 1 pointer

– a few compiler temporaries / return addresses.

• In each call to reverseList (the iterative version you have been using)

– three local pointers (previous, current, next) → constant space.

– no recursion and no dynamic-heap allocations.

Because all of the above are fixed-size items whose total does not grow with n, the auxiliary space required by the whole operation is constant.

Auxiliary space is the extra memory an algorithm uses, not counting the input.

1. **`reverseList(...)`**: The function calls a helper function, `reverseList`, twice. Assuming this is the efficient iterative version that uses three pointers, each call has an auxiliary space complexity of **Θ(1)**.
2. **`Node **current`**: The function declares one local pointer-to-a-pointer for traversal. This variable occupies a constant amount of memory.
3. **Iteration**: The `while` loop reuses the `current` pointer without allocating any new memory.

The total extra memory is the sum of the constant space used by the `reverseList` calls and the constant space for the `current` pointer. The sum of constants is still a constant.

Therefore, the auxiliary space complexity is **Θ(1)**.

$Algorithm$

$TraverseListReverseOrderIterative(START)$

1. $[START\ is\ NULL?]if\ START = NULL, then:$

$Write:``\ List\ is\ Empty.``$

$Return\ and\ Exit$

2. $ReverseList(START)$

$Translation\ of\ ReverseList(START)$

1. $[START\ is\ NULLPTR?]if\ START = NULL, then:$

$Write:``\ Error: List\ is\ Empty.``$

$Return\ and\ exit.$

$[END\ OF\ IF\ STRUCTURE]$

2. $Set\ PREVPTR := NULL$

3. $Set\ CURRPTR := START$

4. $Set\ NEXTPTR := NULL$

5. $Repeat\ while\ CURRPTR\ \neq NULL:$

$Set\ NEXTPTR := LINK[CURRPTR]$

$Set\ LINK[CURRPTR] := PREVPTR$

$Set\ PREVPTR := CURRPTR$

$Set\ CURRPTR := NEXTPTR$

$[END\ OF\ Loop]$

6. $Set\ START := PREVPTR$

---

3. $Set\ CURRPTR := START$

4. $Repeat\ while\ CURRPTR\ \neq NULL:$

$Write: INFO[CURRPTR]$

$SET\ CURRPTR := ADDRESS\ of\ LINK[CURRPTR]$

$[END\ of\ LOOP]$

5. $ReverseList(START)$

$Translation\ of\ ReverseList(START)$

1. $[START\ is\ NULLPTR?]if\ START = NULL, then:$

$Write:``\ Error: List\ is\ Empty.``$

$Return\ and\ exit.$

$[END\ OF\ IF\ STRUCTURE]$

2. $Set\ PREVPTR := NULL$

3. $Set\ CURRPTR := START$

4. $Set\ NEXTPTR := NULL$

5. $Repeat\ while\ CURRPTR\ \neq NULL:$

$Set\ NEXTPTR := LINK[CURRPTR]$

$\boldsymbol{Set\ LINK[CURRPTR] := PREVPTR}$

$\boldsymbol{Set\ PREVPTR := CURRPTR}$

$\boldsymbol{Set\ CURRPTR := NEXTPTR}$

$[\boldsymbol{END\ OF\ Loop}]$

$\boldsymbol{6.\ Set\ START :=\ PREVPTR}$

$[\boldsymbol{END\ of\ Algorithm}\ ]$